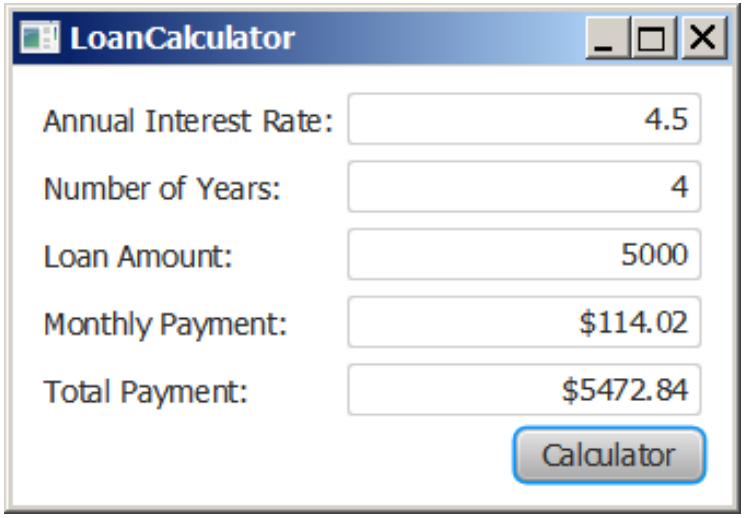
Chapter 15 Event-Driven Programming and Animations

Copyright © 2022 Pearson Education Ltd. All rights reserved.

Motivations

Suppose you want to write a GUI program that lets the user enter a loan amount, annual interest rate, and number of years and click the *Compute Payment* button to obtain the monthly payment and total payment. How do you accomplish the task? You have to use *event-driven programming* to write the code to respond to the button-clicking event.



LoanCalculator

Copyright © 2022 Pearson Education Ltd. All rights reserved.

Objectives

- To get a taste of event-driven programming (§15.1).
- To describe events, event sources, and event classes (§15.2).
- To define handler classes, register handler objects with the source object, and write the code to handle events (§15.3).
- To define handler classes using inner classes (§15.4).
- To define handler classes using anonymous inner classes (§15.5).
- To simplify event handling using lambda expressions (§15.6).
- To develop a GUI application for a loan calculator (§15.7).
- To write programs to deal with **MouseEvent**s (§15.8).
- To write programs to deal with **KeyEvent**s (§15.9).
- To create listeners for processing a value change in an observable object (§15.10).
- To use the **Animation**, **PathTransition**, **FadeTransition**, and **Timeline** classes to develop animations (§15.11).
- To develop an animation for simulating a bouncing ball (§15.12).

Copyright © 2022 Pearson Education Ltd. All rights reserved.

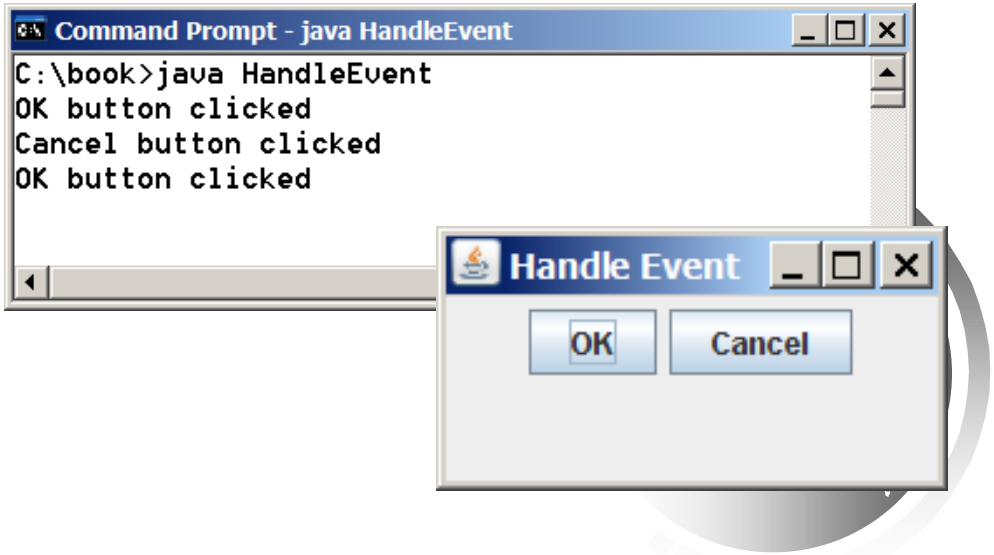
Procedural vs. Event-Driven Programming

- *Procedural programming* is executed in procedural order.
- In event-driven programming, code is executed upon activation of events.

Copyright © 2022 Pearson Education Ltd. All rights reserved.

Taste of Event-Driven Programming

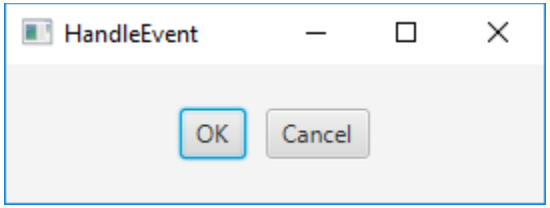
The example displays a button in the frame. A message is displayed on the console when a button is clicked.

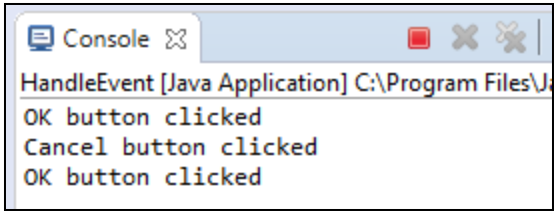


HandleEvent

Copyright © 2022 Pearson Education Ltd. All rights reserved.

```java
import javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.layout.HBox;
import javafx.scene.control.Button;
import javafx.event.ActionEvent;
import javafx.event.EventHandler;
import javafx.geometry.Pos;

public class HandleEvent extends Application {
  public void start(Stage primaryStage) {
    HBox pane = new HBox(10);
    Button btOK = new Button("OK");
    Button btCancel = new Button("Cancel");
    OKHandlerClass handler1 = new OKHandlerClass();
    btOK.setOnAction(handler1);
    CancelHandlerClass handler2 = new CancelHandlerClass();
    btCancel.setOnAction(handler2);
    pane.getChildren().addAll(btOK, btCancel);
    Scene scene = new Scene(pane);
    primaryStage.setScene(scene); primaryStage.show();
}…/*main*/}
class OKHandlerClass implements EventHandler<ActionEvent> {
  @Override
  public void handle(ActionEvent e) {
     System.out.println("OK button clicked");
  }}
class CancelHandlerClass implements EventHandler<ActionEvent> {
  @Override
  public void handle(ActionEvent e) {
    System.out.println("Cancel button clicked");
  }}
```



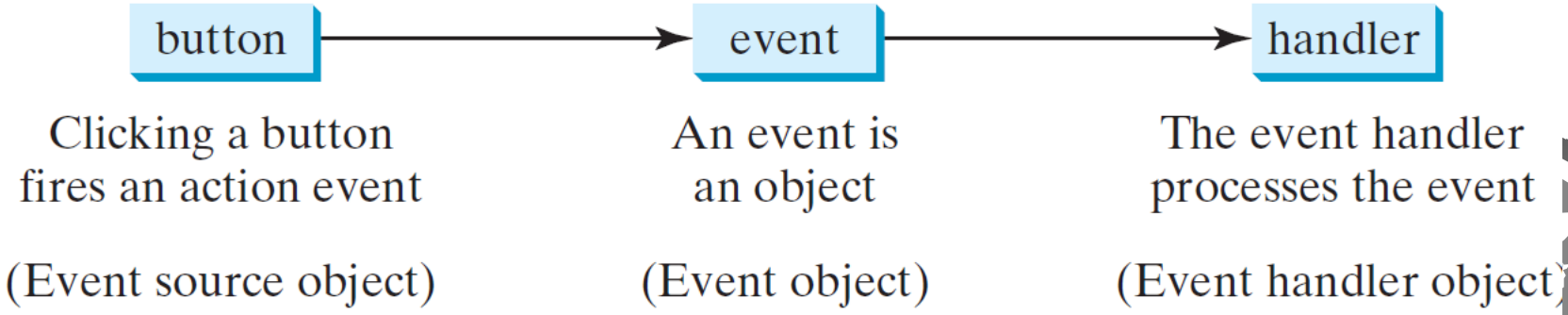


Copyright © 2022 Pearson Education Ltd. All rights reserved.

Handling GUI Events

Source object (e.g., button)

Listener object contains a method for processing the event.



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Handling GUI Events

- The event handler object must be an instance of the **EventHandler<T extends Event>** interface

```
class OKHandlerClass implements EventHandler<ActionEvent>
```

- The **EventHandler** object ***handler*** must be registered with the event source object using the method **source.setOnAction(handler)**.

- The **EventHandler<ActionEvent>** interface contains the **handle(ActionEvent)** method for processing the action event. Your handler class must override this method to respond to the event.

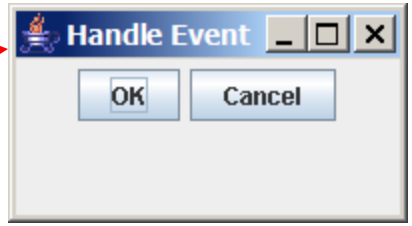
Copyright © 2022 Pearson Education Ltd. All rights reserved.

animation

Trace Execution

```
public class HandleEvent extends Application {
  public void start(Stage primaryStage) {
    …
    OKHandlerClass handler1 = new OKHandlerClass();
    btOK.setOnAction(handler1);
    CancelHandlerClass handler2 = new CancelHandlerClass();
    btCancel.setOnAction(handler2);
    …
    primaryStage.show(); // Display the stage
  }
}

class OKHandlerClass implements EventHandler<ActionEvent> {
  @Override
  public void handle(ActionEvent e) {
    System.out.println("OK button clicked");
  }
}
```

1. Start from the main method to create a window and display it



Copyright © 2022 Pearson Education Ltd. All rights reserved.

animation

Trace Execution

```
public class HandleEvent extends Application {
  public void start(Stage primaryStage) {
    …
    OKHandlerClass handler1 = new OKHandlerClass();
    btOK.setOnAction(handler1);
    CancelHandlerClass handler2 = new CancelHandlerClass();
    btCancel.setOnAction(handler2);
    …
    primaryStage.show(); // Display the stage
  }
}

class OKHandlerClass implements EventHandler<ActionEvent> {
  @Override
  public void handle(ActionEvent e) {
    System.out.println("OK button clicked");
  }
}
```

2. Click OK

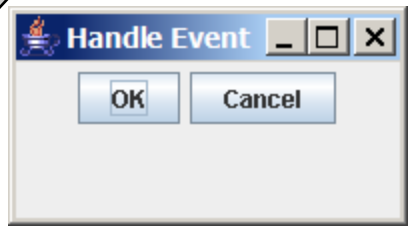
Copyright © 2022 Pearson Education Ltd. All rights reserved.

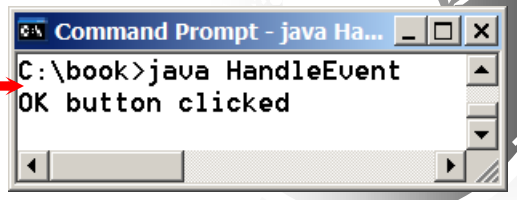
animation

Trace Execution

```
public class HandleEvent extends Application {
  public void start(Stage primaryStage) {
    …
    OKHandlerClass handler1 = new OKHandlerClass();
    btOK.setOnAction(handler1);
    CancelHandlerClass handler2 = new CancelHandlerClass();
    btCancel.setOnAction(handler2);
    …
    primaryStage.show(); // Display the stage
  }
}

class OKHandlerClass implements EventHandler<ActionEvent> {
  @Override
  public void handle(ActionEvent e) {
    System.out.println("OK button clicked");
  }
}
```

3. The JVM invokes the listener's handle method





Copyright © 2022 Pearson Education Ltd. All rights reserved.

How can we use it?

- For each control (button, combo box, etc.):
  - define an event handler
  - construct an instance of event handler
  - tell the control who its event handler is
- Event Handler?
  - code with response to event
  - a.k.a. event listener

Copyright © 2022 Pearson Education Ltd. All rights reserved.

Java's Event Handling

- When the user interacts with a control (source):
  - an *event object* is constructed
  - the event object is sent to all registered *listener objects*
  - the listener object (handler) responds as you defined it to
- Event Listeners (Event Handler)
  - Defined by the application programmer by implementing appropriate interface.

Copyright © 2022 Pearson Education Ltd. All rights reserved.

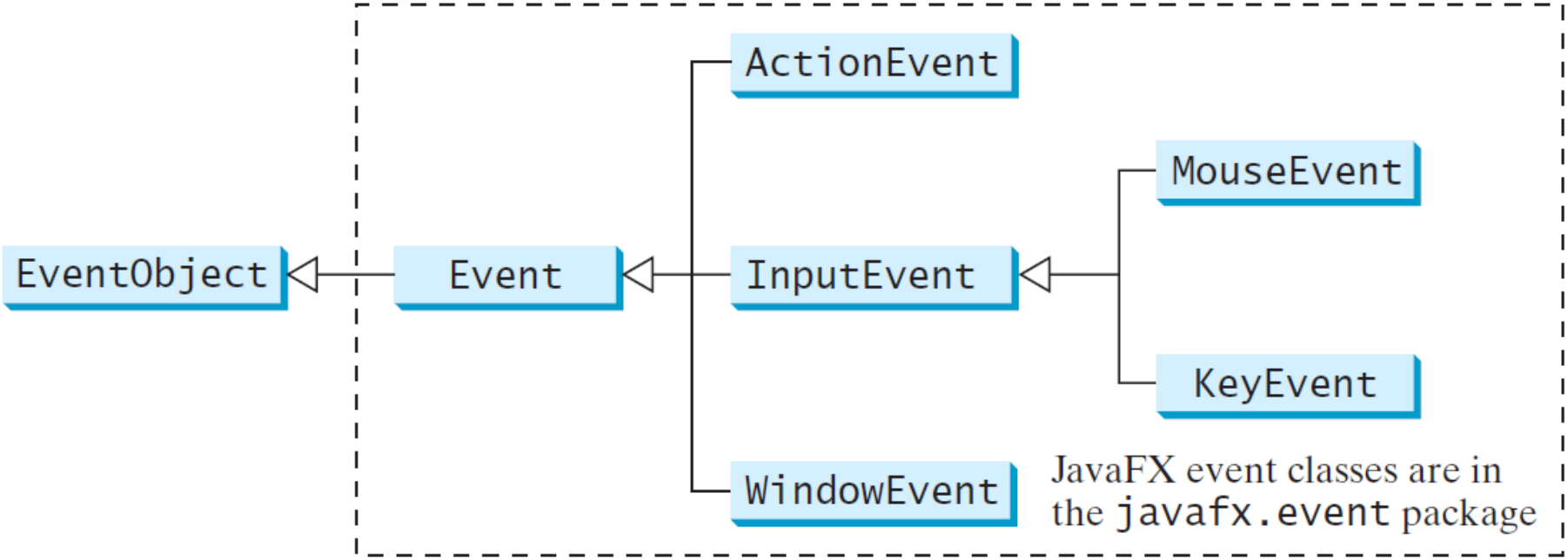
Events

- An *event* can be defined as a type of signal to the program that something has happened.

- The event is generated by external user actions such as mouse movements, mouse clicks, or keystrokes.

Copyright © 2022 Pearson Education Ltd. All rights reserved.

Event Classes



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Event Information

- An event object contains whatever properties are pertinent to the event:
  - the ***source object*** of the event using the **getSource()** instance method in the **EventObject** class.

- The subclasses of **EventObject** deal with special types of events, such as *button actions*, *window events*, *mouse movements*, and *keystrokes*.

Copyright © 2022 Pearson Education Ltd. All rights reserved.

Selected User Actions and Handlers

User Action	*Source Object*	*Event Type Fired*	*Event Registration Method*
Click a button	`Button`	`ActionEvent`	`setOnAction(EventHandler<ActionEvent>)`
Press Enter in a text field	`TextField`	`ActionEvent`	`setOnAction(EventHandler<ActionEvent>)`
Check or uncheck	`RadioButton`	`ActionEvent`	`setOnAction(EventHandler<ActionEvent>)`
Check or uncheck	`CheckBox`	`ActionEvent`	`setOnAction(EventHandler<ActionEvent>)`
Select a new item	`ComboBox`	`ActionEvent`	`setOnAction(EventHandler<ActionEvent>)`
Mouse pressed	`Node, Scene`	`MouseEvent`	`setOnMousePressed(EventHandler<MouseEvent>)`
Mouse released			`setOnMouseReleased(EventHandler<MouseEvent>)`
Mouse clicked			`setOnMouseClicked(EventHandler<MouseEvent>)`
Mouse entered			`setOnMouseEntered(EventHandler<MouseEvent>)`
Mouse exited			`setOnMouseExited(EventHandler<MouseEvent>)`
Mouse moved			`setOnMouseMoved(EventHandler<MouseEvent>)`
Mouse dragged			`setOnMouseDragged(EventHandler<MouseEvent>)`
Key pressed	`Node, Scene`	`KeyEvent`	`setOnKeyPressed(EventHandler<KeyEvent>)`
Key released			`setOnKeyReleased(EventHandler<KeyEvent>)`
Key typed			`setOnKeyTyped(EventHandler<KeyEvent>)`

Copyright © 2022 Pearson Education Ltd. All rights reserved.

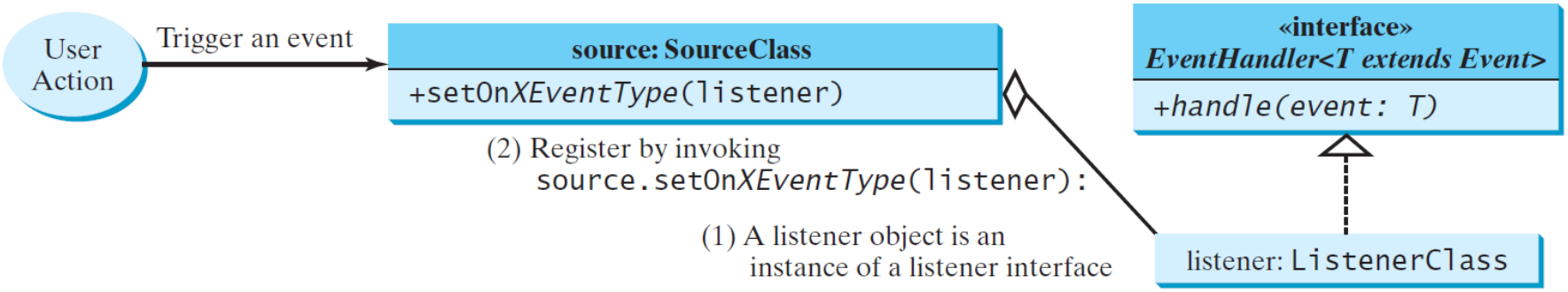
Registering Handlers and Handling Events

- A *handler* is an object that must be registered with an event source object.

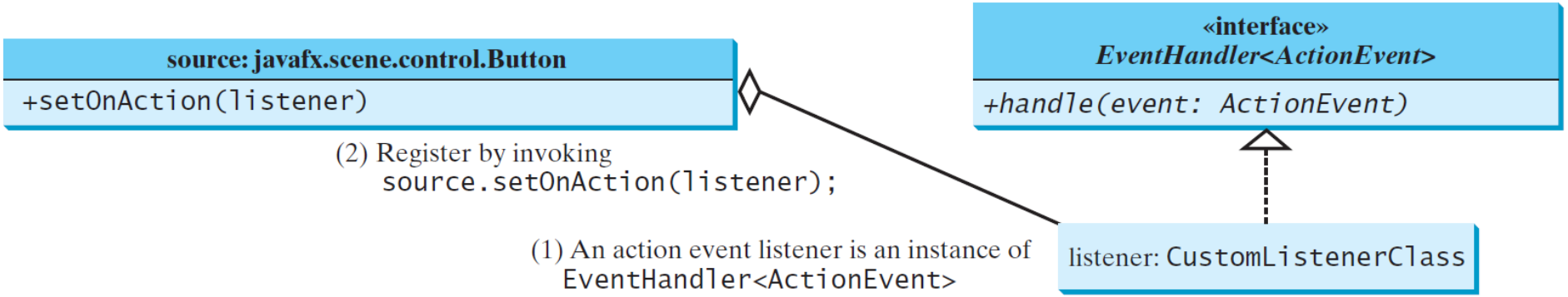
- It must be an instance of an appropriate event-handling interface.

Copyright © 2022 Pearson Education Ltd. All rights reserved.

The Delegation Model



(a) A generic source object with a generic event T



(b) A `Button` source object with an `ActionEvent`

Copyright © 2022 Pearson Education Ltd. All rights reserved.

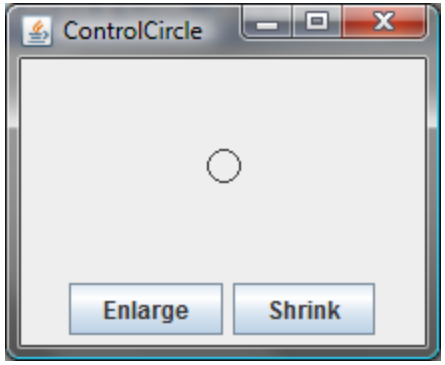
The Delegation Model: Example

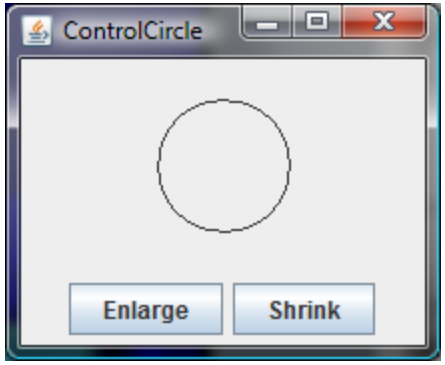
```
Button btOK = new Button("OK");
OKHandlerClass handler = new OKHandlerClass();
btOK.setOnAction(handler);
```


Copyright © 2022 Pearson Education Ltd. All rights reserved.

Example: Second Version for ControlCircle (with listener for Enlarge)

Now let us consider to write a program that uses two buttons to control the size of a circle.

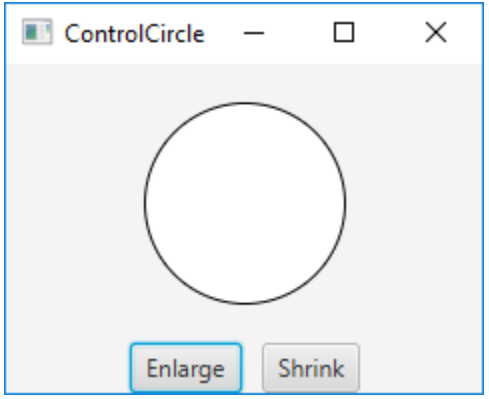


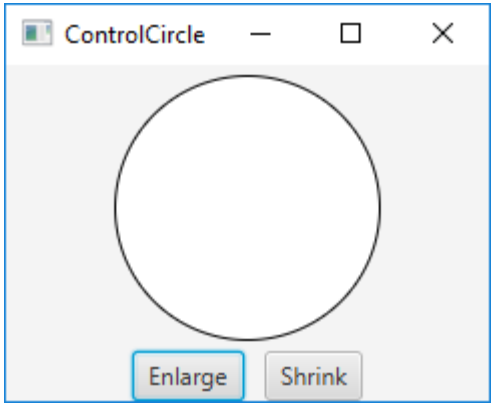


ControlCircle

Copyright © 2022 Pearson Education Ltd. All rights reserved.

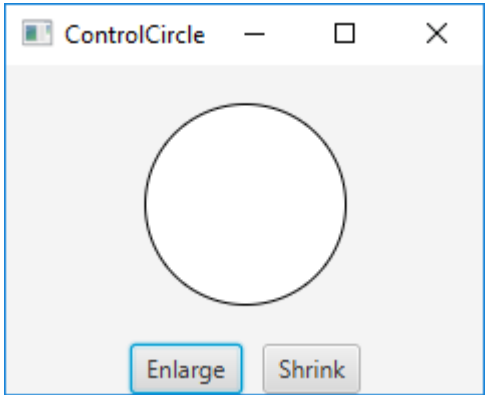
```java
import javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.layout.*;
import javafx.scene.control.Button;
import javafx.event.ActionEvent;
import javafx.event.EventHandler;
import javafx.geometry.Pos;
import javafx.scene.paint.Color;
import javafx.scene.shape.Circle;

public class ControlCircle extends Application {
  private CirclePane circlePane = new CirclePane();
  @Override
  public void start(Stage primaryStage) {
    HBox hBox = new HBox();
    Button btEnlarge = new Button("Enlarge");
    Button btShrink = new Button("Shrink");
    hBox.getChildren().add(btEnlarge);
    hBox.getChildren().add(btShrink);
    btEnlarge.setOnAction(new EnlargeHandler());
    BorderPane borderPane = new BorderPane();
    borderPane.setCenter(circlePane);
    borderPane.setBottom(hBox);
    BorderPane.setAlignment(hBox, Pos.CENTER);
    Scene scene = new Scene(borderPane, 200, 150);
    primaryStage.setScene(scene); primaryStage.show();
  }
```

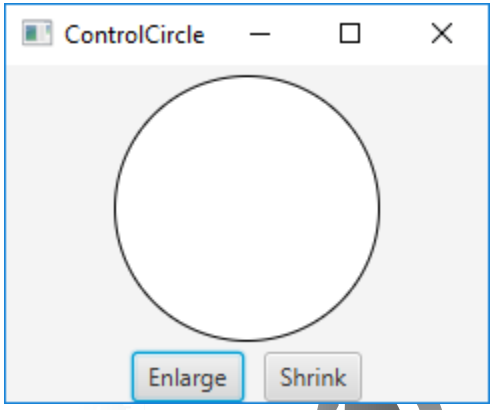




Copyright © 2022 Pearson Education Ltd. All rights reserved.

```
  //inner class
  class EnlargeHandler implements EventHandler<ActionEvent> {
    @Override
    public void handle(ActionEvent e) {
      circlePane.enlarge();
    }
  }
}
class CirclePane extends StackPane {
  private Circle circle = new Circle(50);
  public CirclePane() {
    getChildren().add(circle);
    circle.setStroke(Color.BLACK);
    circle.setFill(Color.WHITE);
  }
  public void enlarge() {
    circle.setRadius(circle.getRadius() + 2);
  }
  public void shrink() {
    circle.setRadius(circle.getRadius() > 2 ?
       circle.getRadius() - 2 : circle.getRadius());
  }
}
```





Copyright © 2022 Pearson Education Ltd. All rights reserved.

Inner Class Listeners

- A listener class is designed specifically to create a listener object for a GUI component (e.g., a button).
  - It will not be shared by other applications.
  - So, it is appropriate to define the listener class inside the frame class as an inner class.

Copyright © 2022 Pearson Education Ltd. All rights reserved.

Inner Classes

- Inner class: A class is a member of another class.
- Advantages: In some applications, you can use an inner class to make programs simple.
- An inner class can reference the data and methods defined in the outer class in which it nests, so you do not need to pass the reference of the outer class to the constructor of the inner class.

Copyright © 2022 Pearson Education Ltd. All rights reserved.

Inner Classes, cont.

```java
public class Test {
  ...
}

public class A {
  ...
}
```

(a)

```java
public class Test {
  ...

  // Inner class
  public class A {
    ...
  }
}
```

(b)

```java
// OuterClass.java: inner class demo
public class OuterClass {
  private int data;

  /** A method in the outer class */
  public void m() {
    // Do something
  }

  // An inner class
  class InnerClass {
    /** A method in the inner class */
    public void mi() {
      // Directly reference data and method
      // defined in its outer class
      data++;
      m();
    }
  }
}
```

(c)

Copyright © 2022 Pearson Education Ltd. All rights reserved.

Inner Classes (cont.)

- Inner classes can make programs simple and concise.
- An inner class supports the work of its containing outer class and is compiled into a class named

  OuterClassName$*InnerClassName*.class.

- For example, the inner class A in Test is compiled into *Test$A*.class .

Copyright © 2022 Pearson Education Ltd. All rights reserved.

Inner Classes (cont.)

- An inner class can be declared public, protected, or private subject to the same visibility rules applied to a member of the class.
- An inner class can be declared static.
  - A static inner class can be accessed using the outer class name.
  - A static inner class cannot access nonstatic members of the outer class

Copyright © 2022 Pearson Education Ltd. All rights reserved.

Inner Classes (cont.)

- Objects of an inner class are often created in the outer class. But you can also create an object of an inner class from another class.

- If the inner class is nonstatic:

```
OuterClass  outerObject = new OuterClass();
OuterClass.InnerClass innerObject = outerObject.new InnerClass();
```

- If the inner class is static:

```
OuterClass.InnerClass innerObject = new OuterClass.InnerClass();
```


Copyright © 2022 Pearson Education Ltd. All rights reserved.

Anonymous Inner Classes

- Inner class listeners can be shortened using anonymous inner classes: inner classes without a name.
  - It combines declaring an inner class and creating an instance of the class in one step.
- An anonymous inner class is declared as follows:

```
new SuperClassName/InterfaceName() {
  // Implement or override methods in superclass/interface
  // Other methods if necessary
}
```


Copyright © 2022 Pearson Education Ltd. All rights reserved.

Anonymous Inner Classes

- An anonymous inner class must always extend a superclass or implement an interface, but it cannot have an explicit extends or implements clause.
- An anonymous inner class must implement all the abstract methods in the superclass or in the interface.
- An anonymous inner class always uses the no-arg constructor from its superclass to create an instance.
- If an anonymous inner class implements an interface, the constructor is Object().
- An anonymous inner class is compiled into a class named OuterClassName$*n*.class.
  - For example, if the outer class Test has two anonymous inner classes, these two classes are compiled into Test$1.class and Test$2.class.

Copyright © 2022 Pearson Education Ltd. All rights reserved.

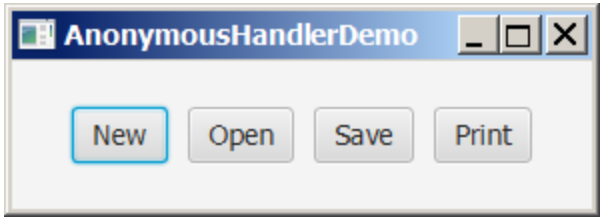
Anonymous Inner Classes (cont.)

```java
public void start(Stage primaryStage) {
  // Omitted

  btEnlarge.setOnAction(
    new EnlargeHandler());
}

class EnlargeHandler
    implements EventHandler<ActionEvent> {
  public void handle(ActionEvent e) {
    circlePane.enlarge();
  }
}
```

(a) Inner class `EnlargeListener`

```java
public void start(Stage primaryStage) {
  // Omitted

  btEnlarge.setOnAction(
    new class EnlargeHandlner
      implements EventHandler<ActionEvent>() {
      public void handle(ActionEvent e) {
        circlePane.enlarge();
      }
    });
}
```

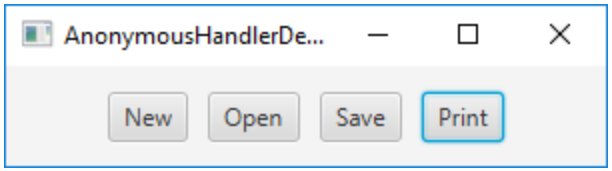
(b) Anonymous inner class

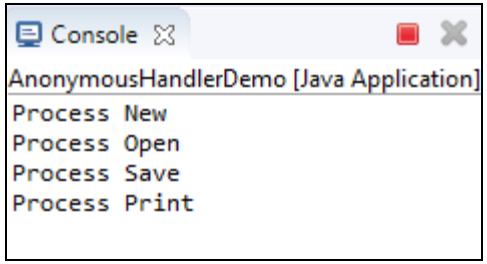


AnonymousHandlerDemo

Copyright © 2022 Pearson Education Ltd. All rights reserved.

```java
import javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.layout.HBox;
import javafx.scene.control.Button;
import javafx.event.ActionEvent;
import javafx.event.EventHandler;
import javafx.geometry.Pos;
public class AnonymousHandlerDemo extends Application {
  public void start(Stage primaryStage) {
    HBox hBox = new HBox();
    Button btNew = new Button("New"); Button btOpen = new Button("Open");
    Button btSave = new Button("Save"); Button btPrint = new Button("Print");
    hBox.getChildren().addAll(btNew, btOpen, btSave, btPrint);
    // Create and register the handler
    btNew.setOnAction(new EventHandler<ActionEvent>() {
      @Override // Override the handle method
      public void handle(ActionEvent e) {
         System.out.println("Process New");
      }
    });
    btOpen.setOnAction(new EventHandler<ActionEvent>() {
      @Override // Override the handle method
      public void handle(ActionEvent e) {
        System.out.println("Process Open");
      }
    });
```





Copyright © 2022 Pearson Education Ltd. All rights reserved.

```
    btSave.setOnAction(new EventHandler<ActionEvent>() {
      @Override // Override the handle method
      public void handle(ActionEvent e) {
        System.out.println("Process Save");
      }
    });

    btPrint.setOnAction(new EventHandler<ActionEvent>() {
      @Override // Override the handle method
      public void handle(ActionEvent e) {
        System.out.println("Process Print");
      }
    });
    Scene scene = new Scene(hBox, 300, 50);
    primaryStage.setTitle("AnonymousHandlerDemo");
    primaryStage.setScene(scene);
    primaryStage.show();
  }
  public static void main(String[] args) {
    launch(args);
  }
}
```


Copyright © 2022 Pearson Education Ltd. All rights reserved.

Simplifying Event Handing Using Lambda Expressions

- *Lambda expression* is a new feature in Java 8.
- Lambda expressions can be viewed as an anonymous class with a concise syntax.
- For example, the following code in (a) can be greatly simplified using a lambda expression in (b) in three lines.

```
btEnlarge.setOnAction(
  new EventHandler<ActionEvent>() {
    @Override
    public void handle(ActionEvent e) {
      // Code for processing event e
    }
  }
});
```

(a) Anonymous inner class event handler

```
btEnlarge.setOnAction(e -> {
  // Code for processing event e
});
```

(b) Lambda expression event handler

Copyright © 2022 Pearson Education Ltd. All rights reserved.

Basic Syntax for a Lambda Expression

- The basic syntax for a lambda expression is either
  - (type1 param1, type2 param2, ...) -> expression

    or
  - (type1 param1, type2 param2, ...) -> { statements; }

- The data type for a parameter may be explicitly declared or implicitly inferred by the compiler. The parentheses can be omitted if there is only one parameter without an explicit data type.

Copyright © 2022 Pearson Education Ltd. All rights reserved.

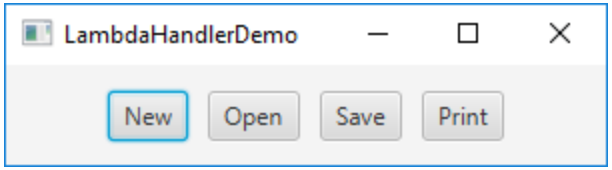
Single Abstract Method Interface (SAM)

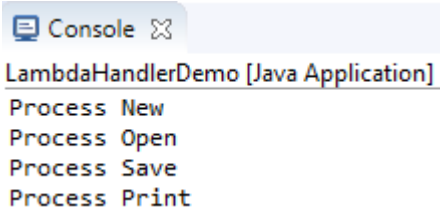
The statements in the lambda expression is all for that method. If it contains multiple methods, the compiler will not be able to compile the lambda expression. So, for the compiler to understand lambda expressions, the interface must contain exactly one abstract method. Such an interface is known as a *functional interface*, or a *Single Abstract Method* (SAM) interface.

AnonymousHandlerDemo

Copyright © 2022 Pearson Education Ltd. All rights reserved.

```java
import javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.layout.HBox;
import javafx.scene.control.Button;
import javafx.event.ActionEvent;
import javafx.event.EventHandler;
import javafx.geometry.Pos;
public class LambdaHandlerDemo extends Application {
  @Override
  public void start(Stage primaryStage) {
    // Hold two buttons in an HBox
    HBox hBox = new HBox();
    hBox.setSpacing(10);
    hBox.setAlignment(Pos.CENTER);
    Button btNew = new Button("New");
    Button btOpen = new Button("Open");
    Button btSave = new Button("Save");
    Button btPrint = new Button("Print");
    hBox.getChildren().addAll(btNew, btOpen, btSave, btPrint);
    btNew.setOnAction((ActionEvent e) -> {System.out.println("Process New");});
    btOpen.setOnAction((e) -> {System.out.println("Process Open");});
    btSave.setOnAction(e -> {System.out.println("Process Save");});
    btPrint.setOnAction(e -> System.out.println("Process Print"));
    Scene scene = new Scene(hBox, 300, 50);
    primaryStage.setScene(scene);
    primaryStage.show();
  }
  public static void main(String[] args) {
  launch(args); }
}
```





Copyright © 2022 Pearson Education Ltd. All rights reserved.

Problem: Loan Calculator

LoanCalculator

Copyright © 2022 Pearson Education Ltd. All rights reserved.

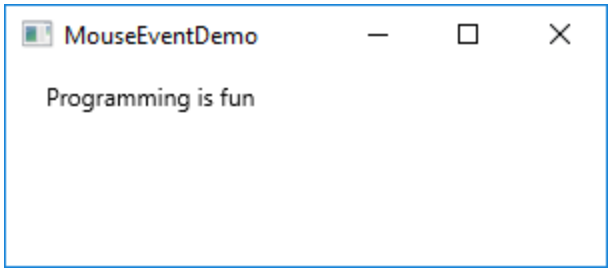
The `MouseEvent` Class

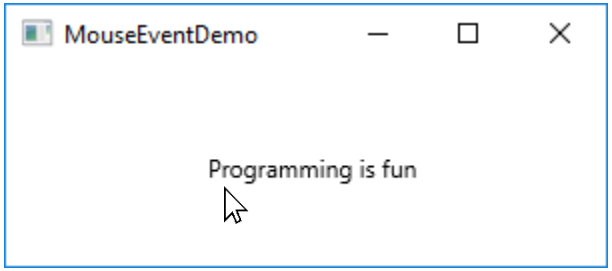
javafx.scene.input.MouseEvent	
`+getButton(): MouseButton`	Indicates which mouse button has been clicked.
`+getClickCount(): int`	Returns the number of mouse clicks associated with this event.
`+getX(): double`	Returns the *x*-coordinate of the mouse point in the event source node.
`+getY(): double`	Returns the *y*-coordinate of the mouse point in the event source node.
`+getSceneX(): double`	Returns the *x*-coordinate of the mouse point in the scene.
`+getSceneY(): double`	Returns the *y*-coordinate of the mouse point in the scene.
`+getScreenX(): double`	Returns the *x*-coordinate of the mouse point in the screen.
`+getScreenY(): double`	Returns the *y*-coordinate of the mouse point in the screen.
`+isAltDown(): boolean`	Returns true if the `Alt` key is pressed on this event.
`+isControlDown(): boolean`	Returns true if the `Control` key is pressed on this event.
`+isMetaDown(): boolean`	Returns true if the mouse `Meta` button is pressed on this event.
`+isShiftDown(): boolean`	Returns true if the `Shift` key is pressed on this event.

MouseEventDemo

Copyright © 2022 Pearson Education Ltd. All rights reserved.

```java
// Move the text with the mouse clicked
import javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.layout.Pane;
import javafx.scene.text.Text;

public class MouseEventDemo extends Application {
  @Override
  public void start(Stage primaryStage) {
    Pane pane = new Pane();
    Text text = new Text(20, 20, "Programming is fun");
    pane.getChildren().addAll(text);
    text.setOnMouseDragged(e -> {
      text.setX(e.getX());
      text.setY(e.getY());
    });
    Scene scene = new Scene(pane, 300, 100);
    primaryStage.setTitle("MouseEventDemo");
    primaryStage.setScene(scene);
    primaryStage.show();
  }
  public static void main(String[] args) {
    launch(args);
  }
}
```





Copyright © 2022 Pearson Education Ltd. All rights reserved.

The `KeyEvent` Class

javafx.scene.input.KeyEvent	
`+getCharacter(): String`	Returns the character associated with the key in this event.
`+getCode(): KeyCode`	Returns the key code associated with the key in this event.
`+getText(): String`	Returns a string describing the key code.
`+isAltDown(): boolean`	Returns true if the `Alt` key is pressed on this event.
`+isControlDown(): boolean`	Returns true if the `Control` key is pressed on this event.
`+isMetaDown(): boolean`	Returns true if the mouse `Meta` button is pressed on this event.
`+isShiftDown(): boolean`	Returns true if the `Shift` key is pressed on this event.

KeyEventDemo

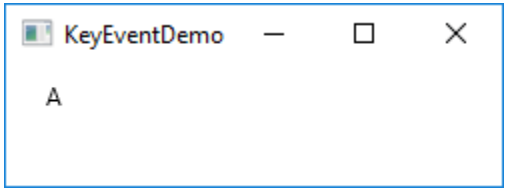
Copyright © 2022 Pearson Education Ltd. All rights reserved.

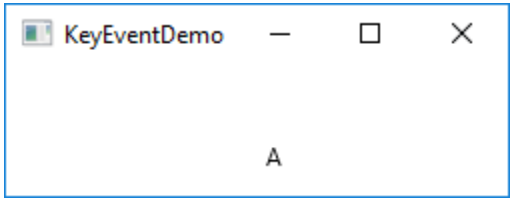
The `KeyCode` Constants

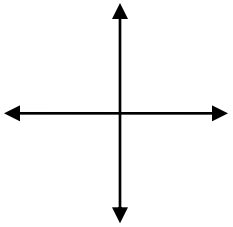
Constant	*Description*	*Constant*	*Description*
`HOME`	The Home key	`CONTROL`	The Control key
`END`	The End key	`SHIFT`	The Shift key
`PAGE_UP`	The Page Up key	`BACK_SPACE`	The Backspace key
`PAGE_DOWN`	The Page Down key	`CAPS`	The Caps Lock key
`UP`	The up-arrow key	`NUM_LOCK`	The Num Lock key
`DOWN`	The down-arrow key	`ENTER`	The Enter key
`LEFT`	The left-arrow key	`UNDEFINED`	The `keyCode` unknown
`RIGHT`	The right-arrow key	`F1` to `F12`	The function keys from F1 to F12
`ESCAPE`	The Esc key	`0` to `9`	The number keys from 0 to 9
`TAB`	The Tab key	`A` to `Z`	The letter keys from A to Z

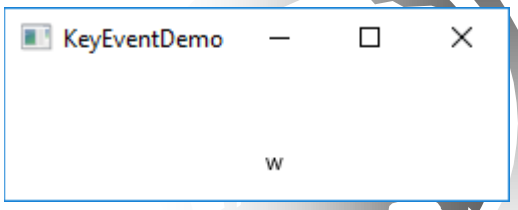
Copyright © 2022 Pearson Education Ltd. All rights reserved.

```java
import javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.layout.Pane;
import javafx.scene.text.Text;

public class KeyEventDemo extends Application {
  @Override
  public void start(Stage primaryStage) {
    Pane pane = new Pane();
    Text text = new Text(20, 20, "A");
    text.setFocusTraversable(true);
    pane.getChildren().add(text);
    text.setOnKeyPressed(e -> {
      switch (e.getCode()) {
        case DOWN: text.setY(text.getY() + 10); break;
        case UP: text.setY(text.getY() - 10); break;
        case LEFT: text.setX(text.getX() - 10); break;
        case RIGHT: text.setX(text.getX() + 10); break;
        default:
          if (Character.isLetterOrDigit(e.getText().charAt(0)))
            text.setText(e.getText());
      }
    });
    Scene scene = new Scene(pane);
    primaryStage.setTitle("KeyEventDemo");
    primaryStage.setScene(scene);
    primaryStage.show();

  } …/*main*/}
```









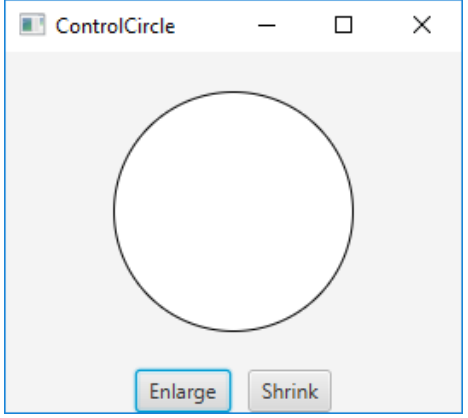
Copyright © 2022 Pearson Education Ltd. All rights reserved.

Example: Control Circle with Mouse and Key

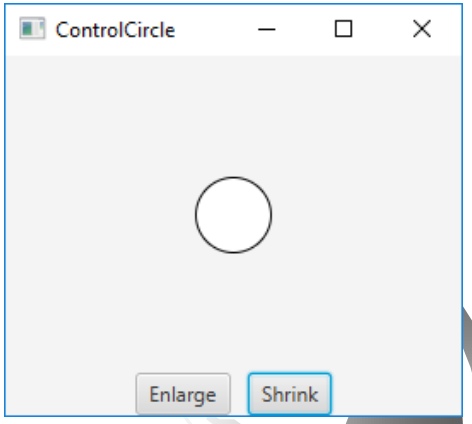
ControlCircleWithMouseAndKey

Copyright © 2022 Pearson Education Ltd. All rights reserved.

```java
// several import statements …
public class ControlCircleWithMouseAndKey extends Application {
  private CirclePane circlePane = new CirclePane();
  @Override
  public void start(Stage primaryStage) {
    HBox hBox = new HBox();
    hBox.setSpacing(10);
    hBox.setAlignment(Pos.CENTER);
    Button btEnlarge = new Button("Enlarge");
    Button btShrink = new Button("Shrink");
    hBox.getChildren().add(btEnlarge);
    hBox.getChildren().add(btShrink);
    btEnlarge.setOnAction(e -> circlePane.enlarge());
    btShrink.setOnAction(e -> circlePane.shrink());
    circlePane.setOnMouseClicked(e -> {
      if (e.getButton() == MouseButton.PRIMARY) {
        circlePane.enlarge();
      }
      else if (e.getButton() == MouseButton.SECONDARY) {
        circlePane.shrink();
      }
    });
    circlePane.setFocusTraversable(true);
    circlePane.setOnKeyPressed(e -> {
      if (e.getCode() == KeyCode.U) {
        circlePane.enlarge();
      }
      else if (e.getCode() == KeyCode.D) {
        circlePane.shrink();
      }
    });
    BorderPane borderPane = new BorderPane();
    borderPane.setCenter(circlePane);
    borderPane.setBottom(hBox);
    BorderPane.setAlignment(hBox, Pos.CENTER);
    Scene scene = new Scene(borderPane, 200, 150);
    …
```



The circle will enlarge by
- clicking Enlarge button,
- Mouse left click,
- Keyboard U press.



The circle will shrink by
- clicking Shrink button,
- Mouse right click,
- Keyboard D press.

Copyright © 2022 Pearson Education Ltd. All rights reserved.

Listeners for Observable Objects

- You can add a listener to process a value change in an observable object.
- An instance of **Observable** is known as an *observable object*, which contains the **addListener(InvalidationListener listener)** method for adding a listener.
- Once the value is changed in the property, a listener is notified.
- The listener class should implement the **InvalidationListener** interface, which uses the **invalidated(Observable o)** method to handle the property value change. Every binding property is an instance of **Observable**.

ObservablePropertyDemo

DisplayResizableClock

Copyright © 2022 Pearson Education Ltd. All rights reserved.

// ObservablePropertyDemo.java

```
import javafx.beans.InvalidationListener;
import javafx.beans.Observable;
import javafx.beans.property.DoubleProperty;import
javafx.beans.property.SimpleDoubleProperty;

public class ObservablePropertyDemo {
  public static void main(String[] args) {
    DoubleProperty balance = new SimpleDoubleProperty();
    balance.addListener(new InvalidationListener() {
      public void invalidated(Observable ov) {
        System.out.println("The new value is " +
           balance.doubleValue());
      }
    });
    balance.set(4.5);
  }
}
```

Output:

The new value is 4.5

Copyright © 2022 Pearson Education Ltd. All rights reserved.

Animation

JavaFX provides the **Animation** class with the core functionality for all animations.

The getter and setter methods for property values and a getter for property itself are provided in the class, but omitted in the UML diagram for brevity.

javafx.animation.Animation	
-autoReverse: BooleanProperty	Defines whether the animation reverses direction on alternating cycles.
-cycleCount: IntegerProperty	Defines the number of cycles in this animation.
-rate: DoubleProperty	Defines the speed and direction for this animation.
-status: ReadOnlyObjectProperty <Animation.Status>	Read-only property to indicate the status of the animation.
+pause(): void	Pauses the animation.
+play(): void	Plays the animation from the current position.
+stop(): void	Stops the animation and resets the animation.

Copyright © 2022 Pearson Education Ltd. All rights reserved.

PathTransition

The **PathTransition** class animates the moves of a node along a path from one end to the other over a given time.

The getter and setter methods for property values and a getter for property itself are provided in the class, but omitted in the UML diagram for brevity.

javafx.animation.PathTransition

```
-duration: ObjectProperty<Duration>
-node: ObjectProperty<Node>
-orientation: ObjectProperty
   <PathTransition.OrientationType>
-path: ObjectType<Shape>
```

```
+PathTransition()
+PathTransition(duration: Duration,
   path: Shape)
+PathTransition(duration: Duration,
   path: Shape, node: Node)
```

The duration of this transition.	
The target node of this transition.	
The orientation of the node along the path.	
The shape whose outline is used as a path to animate the node move.	
Creates an empty `PathTransition`.	
Creates a `PathTransition` with the specified duration and path.	
Creates a `PathTransition` with the specified duration, path, and node.	

PathTransitionDemo

FlagRisingAnimation

Copyright © 2022 Pearson Education Ltd. All rights reserved.

FadeTransition

The **FadeTransition** class animates the change of the opacity in a node over a given time.

The getter and setter methods for property values and a getter for property itself are provided in the class, but omitted in the UML diagram for brevity.

javafx.animation.FadeTransition

```
-duration: ObjectProperty<Duration>
-node: ObjectProperty<Node>
-fromValue: DoubleProperty
-toValue: DoubleProperty
-byValue: DoubleProperty
```

```
+FadeTransition()
+FadeTransition(duration: Duration)
+FadeTransition(duration: Duration,
   node: Node)
```

The duration of this transition.	
The target node of this transition.	
The start opacity for this animation.	
The stop opacity for this animation.	
The incremental value on the opacity for this animation.	
Creates an empty `FadeTransition`.	
Creates a `FadeTransition` with the specified duration.	
Creates a `FadeTransition` with the specified duration and node.	

FadeTransitionDemo

Copyright © 2022 Pearson Education Ltd. All rights reserved.

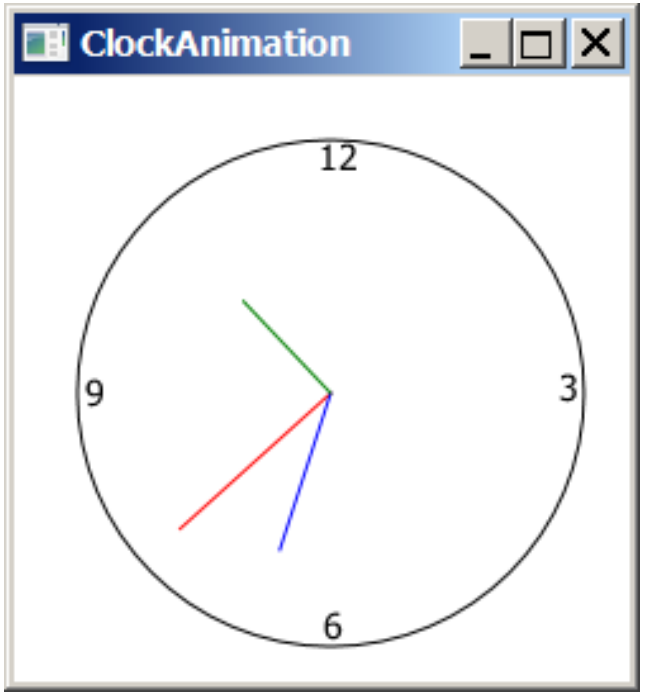
Timeline

- **PathTransition** and **FadeTransition** define specialized animations.
- The **Timeline** class can be used to program any animation using one or more **KeyFrame**s.
  - **KeyFrame** defines target values at a specified point in time for a set of variables that are interpolated along a **Timeline**.
  - Each **KeyFrame** is executed sequentially at a specified time interval.
- **Timeline** inherits from **Animation**.
- You can construct a **Timeline** using the constructor .
  `new Timeline(KeyFrame... keyframes).`
- A **KeyFrame** can be constructed using
  `new KeyFrame(Duration duration, EventHandler<ActionEvent> onFinished)`
  - The handler **onFinished** is called when the duration for the key frame is elapsed.

TimelineDemo Run

Copyright © 2022 Pearson Education Ltd. All rights reserved.

Clock Animation



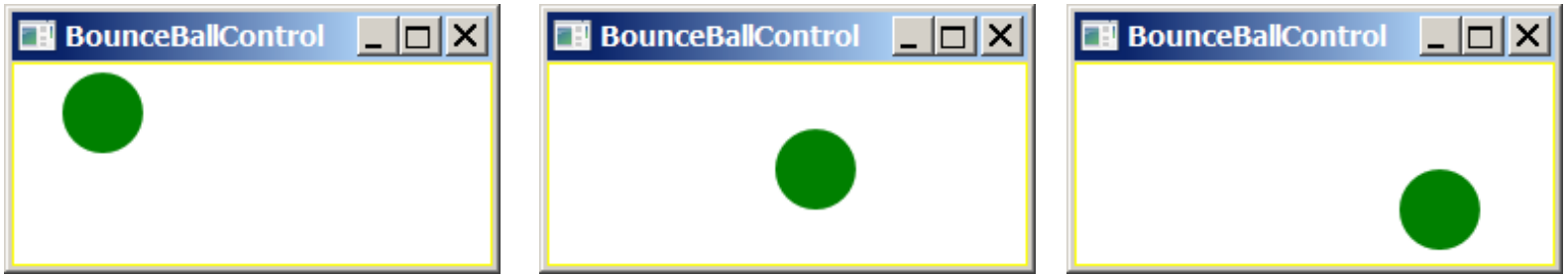
ClockAnimation

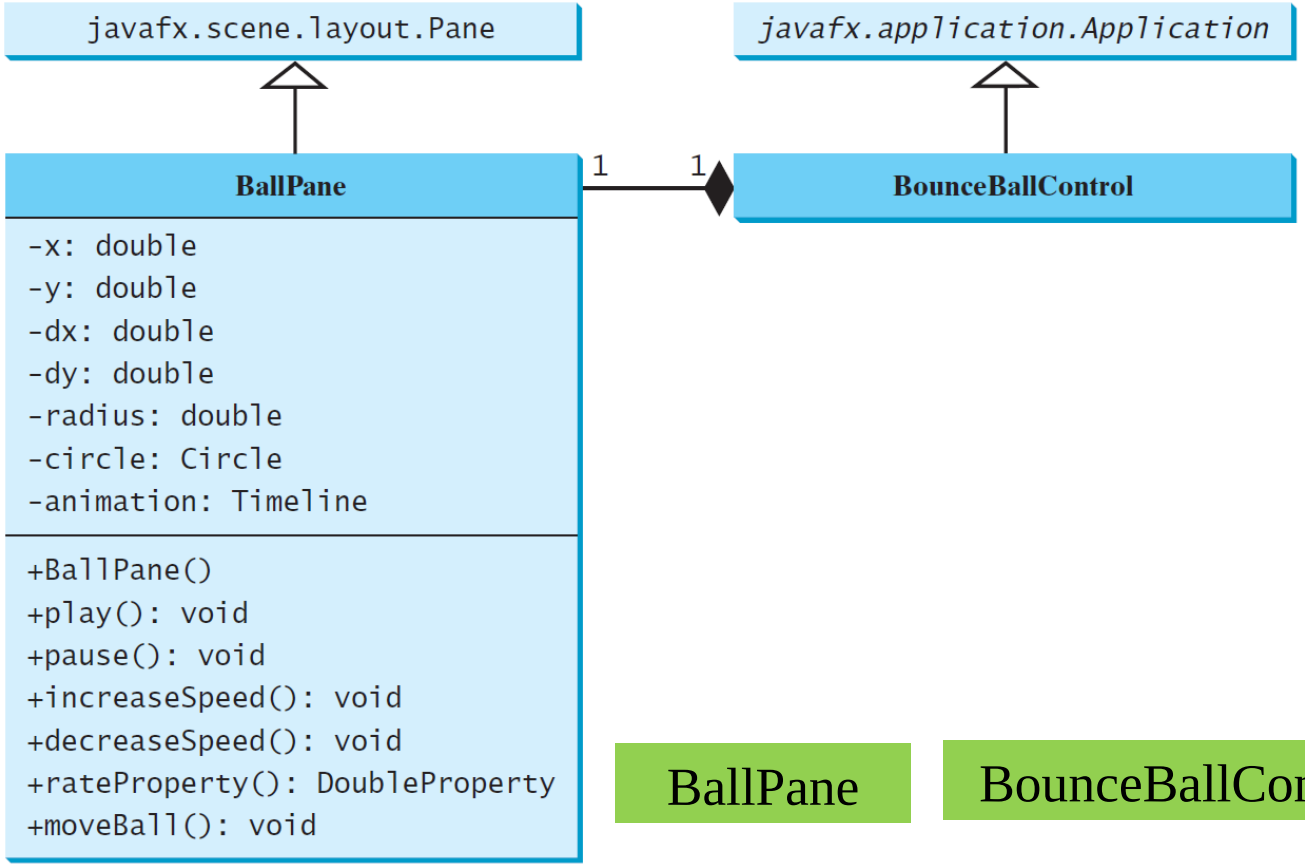
Copyright © 2022 Pearson Education Ltd. All rights reserved.

```java
import javafx.application.Application;
import javafx.stage.Stage;
import javafx.animation.KeyFrame;
import javafx.animation.Timeline;
import javafx.event.ActionEvent;
import javafx.event.EventHandler;
import javafx.scene.Scene;
import javafx.util.Duration;
public class ClockAnimation extends Application {
  @Override // Override the start method in the Application class
  public void start(Stage primaryStage) {
    ClockPane clock = new ClockPane(); // Create a clock

    // Create a handler for animation
    EventHandler<ActionEvent> eventHandler = e -> {
      clock.setCurrentTime(); // Set a new clock time
    };

    // Create an animation for a running clock
    Timeline animation = new Timeline(
      new KeyFrame(Duration.millis(1000), eventHandler));
    animation.setCycleCount(Timeline.INDEFINITE);
    animation.play(); // Start animation

    // Create a scene and place it in the stage
    Scene scene = new Scene(clock, 250, 250);
    primaryStage.setTitle("ClockAnimation"); // Set the stage title
    primaryStage.setScene(scene); // Place the scene in the stage
    primaryStage.show(); // Display the stage
  }

 public static void main(String[] args) {
    launch(args);
  }
}
```


Copyright © 2022 Pearson Education Ltd. All rights reserved.

Case Study: Bouncing Ball





BallPane

BounceBallControl

Copyright © 2022 Pearson Education Ltd. All rights reserved.